

Australian Automotive Parts E-Commerce Platform

Comprehensive Project Breakdown & Development Guide

PROJECT OVERVIEW

What Will Be Built

A **Progressive Web Application (PWA)** e-commerce platform specifically designed for the Australian automotive aftermarket, featuring:

- **New automotive parts** (engine components, transmission parts, body panels, electrical systems, etc.)
- **Used automotive parts** (condition-based pricing and descriptions)
- **Vehicle sales** (complete cars, trucks, motorcycles)
- Mobile-optimized responsive design
- Installable to home screen (PWA functionality)
- Secure payment processing for Australian market
- Full shopping cart and checkout experience

Technology Stack

Component	Technology	Why This Choice
Frontend	Next.js 14+ (React)	SEO-friendly, fast, modern framework
Backend	Next.js API Routes	Integrated backend, no separate server needed
Database	PostgreSQL	Robust, free, scalable, supports complex queries
Payments	Stripe	Lowest fees in Australia (1.75% + 30¢)
Authentication	NextAuth.js	Industry standard, secure, easy integration
Styling	Tailwind CSS	Rapid development, responsive design
Hosting	Vercel / Railway	Easy deployment, automatic scaling
Image Storage	Cloudinary / S3	CDN delivery, optimization

What's Included

- ✓ Product catalog with categories and search
- ✓ Shopping cart and checkout flow
- ✓ Stripe payment integration (Australian configuration)
- ✓ User accounts and order history
- ✓ Admin panel for product/order management
- ✓ Mobile-responsive design
- ✓ PWA functionality (installable)
- ✓ Email notifications
- ✓ Inventory tracking
- ✓ Condition selector (New/Used)

What's Excluded (But Can Be Added Later)

- ✗ Native iOS/Android apps (PWA only)
 - ✗ Advanced vehicle compatibility database (Phase 5)
 - ✗ Automated inventory sync with suppliers (requires n8n)
 - ✗ Live chat support
 - ✗ Multi-currency support
 - ✗ Shipping integrations (Aus Post API)
 - ✗ eBay/Gumtree cross-posting
-

ARCHITECTURE DECISIONS

Should You Use n8n?

What is n8n?

n8n is a **workflow automation tool** (like Zapier but self-hosted) that connects different services together. For example:

- When an order is placed → send SMS to warehouse
- When inventory drops below 5 → email supplier
- Daily export of orders → Google Sheets
- Sync inventory between your store and eBay

When It Makes Sense

- ✓ You need complex automations (sync inventory across multiple platforms)
- ✓ You have external systems to integrate (ERP, warehouse management)
- ✓ You want to automate repetitive tasks (daily reports, low stock alerts)
- ✓ You're comfortable managing another service

When It Doesn't Make Sense

- ✗ You're just starting out (adds complexity)
- ✗ Basic e-commerce is all you need
- ✗ You don't have external systems to integrate
- ✗ Budget is tight (hosting costs + learning curve)

Recommendation

Start without n8n. Build your core platform first. Once you're processing orders regularly and identify repetitive tasks, then add n8n for automation. You can integrate it later without rebuilding anything.

When to revisit: After Phase 4 (Admin Panel), if you find yourself manually doing the same tasks daily.

Should You Use External Databases (Baserow/Airtable)?

What Are These?

Tools like **Baserow** and **Airtable** are visual spreadsheet-database hybrids where you can manually edit data through a nice interface.

Pros of External Databases

- ✓ Easy manual editing (like Excel)
- ✓ Non-technical staff can manage products
- ✓ Visual interface for data management
- ✓ Quick setup for prototyping

Cons of External Databases

- ✗ **API rate limits** (slow down your site under traffic)
- ✗ **Performance issues** (extra network requests for every query)
- ✗ **Monthly costs** (\$20-\$100/month for decent limits)
- ✗ **Harder to scale** (can't handle 1000s of concurrent users)

✗ **Complex queries are difficult** (filtering, sorting, joins)

✗ **Dependency risk** (service goes down = your site breaks)

Recommendation

Use PostgreSQL directly. It's included in most hosting platforms (free), faster, more powerful, and scales better. You'll build an admin panel anyway (Phase 4) which gives you the same "easy editing" benefit without the downsides.

If you need visual editing: Build a good admin panel. It's a better investment than paying for external services.

Cheapest Checkout Strategy

The Hard Truth

There's no way to avoid payment gateway fees if you want to accept online payments legally and securely in Australia.

Australian Payment Options Comparison

Provider	Fees	Setup	Pros	Cons
Stripe	1.75% + 30¢	Free	Easy integration, best developer experience, instant setup	-
PayPal	2.6% + 30¢	Free	Trusted brand, buyers may already have accounts	Higher fees, clunky integration
Square	1.9% + 30¢	Free	Good for in-person + online	Less features for pure e-commerce
Pin Payments	1.75% + 30¢	Free	Australian company	Smaller, less documentation
Direct Bank Transfer	Free	Manual	No fees	Manual reconciliation, slow, poor UX

Recommendation

Use Stripe.

- Lowest fees for Australian online businesses (1.75% + 30¢)
- Best documentation and developer tools
- Handles fraud protection, PCI compliance
- Supports Australian cards, direct debits, BPAY (later)
- No monthly fees (only per-transaction)

Math example:

- \$100 sale = \$1.75 + \$0.30 = \$2.05 fee (you receive \$97.95)
- \$500 sale = \$8.75 + \$0.30 = \$9.05 fee (you receive \$490.95)

What About “Open Source” Payment Solutions?

Even open-source shopping carts still require a payment gateway (Stripe, PayPal, etc.) to process cards. You can't avoid these fees unless you only accept bank transfers (terrible UX, slow).

PHASED DEVELOPMENT PLAN

PHASE 1: Core Foundation

Goal: Get the basic app structure running with authentication and database

Estimated Credits: 800-1,200

Estimated Time: 2-3 sessions

Tasks

1. Project Setup & Configuration

- Initialize Next.js project with TypeScript
- Configure Tailwind CSS
- Set up folder structure (app, components, lib, utils)
- Environment variables setup

2. Database Schema Design

sql

Tables needed:

- users (id, email, password_hash, name, role, created_at)
- categories (id, name, slug, parent_id, description)
- products (id, title, description, price, sku, condition, category_id, stock, images, created_at)
- cart_items (id, user_id, product_id, quantity)
- orders (id, user_id, total, status, stripe_payment_id, created_at)
- order_items (id, order_id, product_id, quantity, price_at_purchase)
- addresses (id, user_id, type, street, city, state, postcode, is_default)

3. Authentication System

- NextAuth.js setup with credentials provider
- Registration page (email, password, name)
- Login page
- Protected routes middleware
- Password hashing (bcrypt)
- Session management

4. Basic Layout & Navigation

- Header with logo, search bar, cart icon, account menu
- Footer with links (About, Contact, Terms, Privacy)
- Responsive mobile menu
- Loading states and error handling

Deliverables:

- Working authentication (signup/login)
 - Database connected and schema created
 - Basic responsive layout
-

PHASE 2: Product Management (Frontend)

Goal: Build the customer-facing product browsing experience

Estimated Credits: 600-900

Estimated Time: 2 sessions

Tasks

1. Category System

- Categories for Parts:
 - Engine Components (pistons, gaskets, belts, filters)
 - Transmission & Drivetrain (clutches, gearboxes, driveshafts)
 - Suspension & Brakes (shocks, struts, brake pads, rotors)
 - Electrical & Lighting (alternators, starters, headlights)
 - Body & Exterior (panels, bumpers, mirrors, grilles)
 - Interior & Accessories (seats, carpets, steering wheels)
- Categories for Vehicles:
 - Cars (sedans, SUVs, hatchbacks)
 - Trucks & Utes (4x4s, commercial vehicles)
 - Motorcycles (road bikes, dirt bikes, scooters)

2. Product Listing Pages

- Grid layout with product cards
- Product card shows: image, title, price, condition badge, stock status
- Pagination (20-50 products per page)
- Loading skeletons
- Empty state handling

3. Filtering & Sorting

- Filter by category
- Filter by condition (New/Used)
- Filter by price range (slider)
- Sort by: Price (low-high), Price (high-low), Newest, Popular
- Filter by stock availability

4. Search Functionality

- Search bar in header
- Full-text search across product titles and descriptions
- Search results page
- No results state with suggestions

5. Product Detail Page

- Image gallery (main image + thumbnails, zoom on hover)
- Product title, SKU, condition badge

- Price (large, prominent)
- Stock status (In Stock / Low Stock / Out of Stock)
- Description (rich text)
- Specifications table (for parts: compatibility, dimensions, weight)
- Add to cart button (quantity selector)
- Breadcrumb navigation

Deliverables:

- Browse products by category
 - Search and filter products
 - View detailed product pages
 - Condition indicator (New/Used) throughout
-

PHASE 3: Shopping Cart & Checkout

Goal: Enable customers to purchase products

Estimated Credits: 700-1,000

Estimated Time: 2-3 sessions

Tasks

1. Shopping Cart Functionality

- Add to cart (with quantity)
- Cart page showing all items
- Update quantity (+ and - buttons)
- Remove items
- Subtotal calculation
- GST calculation (10% in Australia)
- Cart persists across sessions
- Cart badge in header (item count)
- Empty cart state

2. Checkout Flow

- **Step 1:** Shipping address form
 - Address validation for Australian postcodes
 - Save address to account (optional)
 - Use saved address (for returning customers)
- **Step 2:** Shipping method selection (future: integrate Aus Post rates)
- **Step 3:** Order review
 - Item summary
 - Shipping address confirmation
 - Total breakdown (subtotal, shipping, GST, total)

3. Stripe Payment Integration

- Stripe account setup (Australian configuration)
- Stripe Checkout integration (hosted payment page - easiest)
- Create payment intent on order submission
- Handle successful payment webhook
- Handle failed payment

- Redirect to order confirmation
- Store payment ID in database

4. **Order Confirmation**

- Thank you page with order number
- Order summary
- Estimated delivery date
- Print order button
- Email confirmation trigger

5. **Email Notifications**

- Order confirmation email (Nodemailer + Gmail SMTP or SendGrid)
- Email template with order details
- Send to customer email

Deliverables:

- Working shopping cart
- Complete checkout flow
- Stripe payment processing
- Order confirmation page and email

 **STOPPING POINT #1:** After Phase 3, you have a **functional e-commerce store**. Customers can browse, purchase, and receive confirmations. You can manually manage products via database tools.

PHASE 4: Admin Panel

Goal: Build backend interface for managing store

Estimated Credits: 800-1,200

Estimated Time: 2-3 sessions

Tasks

1. **Admin Dashboard**

- Admin authentication (role-based access)
- Overview page with metrics:
 - Total revenue (today, week, month)
 - Order count
 - Low stock alerts
 - Recent orders list
 - Charts (orders over time, revenue trends)

2. **Product CRUD**

- Products list page (searchable, sortable table)
- Add new product form:
 - Title, description, SKU
 - Category dropdown
 - Condition dropdown (New/Used)
 - Price, stock quantity
 - Multiple image upload (drag & drop)

- Specifications (dynamic fields)
- Edit product (same form, pre-filled)
- Delete product (with confirmation)
- Bulk actions (delete, update stock)

3. Order Management

- Orders list page (filterable by status)
- Order detail view:
 - Customer info
 - Items ordered
 - Payment status
 - Shipping address
 - Update order status:
 - Pending → Processing → Shipped → Delivered → Cancelled
 - Print invoice/packing slip
 - Refund order (Stripe refund API)

4. Inventory Tracking

- Stock level warnings (low stock badge)
- Automatic stock reduction on order
- Stock adjustment form (manual add/remove)
- Inventory history log

5. User Management

- Users list (all customers)
- View user details (orders, addresses)
- Ban/unban users
- Reset passwords

Deliverables:

- Full admin dashboard
- Manage products without touching database
- Process and track orders
- Monitor inventory

 **STOPPING POINT #2:** After Phase 4, you have a **fully manageable store** without needing database tools. This is ideal for ongoing operations.

PHASE 5: Advanced Features

Goal: Add features that improve customer experience and drive sales

Estimated Credits: 500-800 per feature

Estimated Time: 1-2 sessions per feature

Feature 5A: User Profiles & Order History

- Customer dashboard
- Order history (view past orders)
- Track order status

- Saved addresses management
- Account settings (change password, email)

Feature 5B: Reviews & Ratings

- Leave review on purchased products
- Star rating (1-5)
- Helpful vote system
- Admin moderation (approve/reject reviews)
- Display average rating on product pages

Feature 5C: Wishlist

- Add to wishlist button
- Wishlist page
- Move from wishlist to cart
- Share wishlist (unique URL)

Feature 5D: Advanced Search & Filters

- Faceted search (filter by multiple attributes)
- Filter by vehicle compatibility (“fits 2015 Toyota Hilux”)
- Filter by brand/manufacturer
- Price history (show if product is on sale)
- Related products suggestions

Feature 5E: Vehicle Compatibility Checker

- Database of vehicles (make, model, year)
- Products tagged with compatible vehicles
- “Select your vehicle” tool (dropdowns: make → model → year)
- Filter products by selected vehicle
- “Fits your vehicle” badge on products
- **Note:** This is complex and requires extensive data

Deliverables (choose what matters most):

- Pick 2-3 features from this phase based on priorities
- Each feature is independently valuable

PHASE 6: PWA & Mobile Optimization

Goal: Make the app installable and work offline

Estimated Credits: 400-600

Estimated Time: 1 session

Tasks

1. PWA Configuration

- `manifest.json` file (app name, icons, colors)
- Service worker registration
- Cache strategies (cache-first for static assets, network-first for dynamic data)
- App icons (multiple sizes: 192x192, 512x512)

2. Install Prompts

- “Add to Home Screen” prompt (iOS)
- Install banner (Android)
- Deferred install prompt (show after user engagement)

3. Offline Functionality

- Offline page (show when no internet)
- Cache product pages for offline browsing
- Show offline indicator
- Sync cart when back online

4. Mobile UI Polish

- Touch-friendly buttons (minimum 44x44px)
- Swipe gestures (swipe to remove from cart)
- Bottom navigation (mobile-friendly)
- Optimized forms (correct keyboard types)
- Fast transitions and animations

Deliverables:

- Installable PWA (add to home screen)
- Works offline (basic functionality)
- Polished mobile experience

 **STOPPING POINT #3:** After Phase 6, you have a **full-featured PWA** that feels like a native app.

PHASE 7: Polish & Deployment

Goal: Final touches and production launch

Estimated Credits: 300-500

Estimated Time: 1-2 sessions

Tasks

1. Bug Fixes & Testing

- Cross-browser testing (Chrome, Safari, Firefox)
- Mobile device testing (iOS Safari, Android Chrome)
- Fix any UI/UX issues
- Payment flow testing (test mode)

2. Performance Optimization

- Image optimization (Next.js Image component)
- Lazy loading for images
- Code splitting
- Minimize bundle size
- Database query optimization (indexes)
- Lighthouse score improvement (aim for 90+)

3. SEO Optimization

- Meta tags (title, description) for all pages
- Open Graph tags (for social sharing)
- Structured data (Product schema)

- XML sitemap
- robots.txt
- Google Analytics setup

4. **Production Deployment**

- Deploy to Vercel (or Railway)
- Configure custom domain
- SSL certificate (automatic with Vercel)
- Database migration to production
- Environment variables setup
- Stripe production keys

5. **Documentation**

- Admin user guide
- How to add products
- How to process orders
- Troubleshooting guide

Deliverables:

- Live, production-ready e-commerce platform
- Optimized performance
- SEO-ready
- Admin documentation

TOTAL ESTIMATED CREDITS

Minimum Viable Product (MVP)

Phases 1-3: 2,100 - 3,100 credits

- Core functionality: browse, search, purchase
- Manual product management (via database)
- Good for: Testing the market, first customers

Full-Featured Store

Phases 1-6: 4,300 - 6,200 credits

- Everything in MVP plus:
- Admin panel (easy management)
- Advanced features (reviews, profiles, etc.)
- PWA functionality
- Good for: Ongoing business operations

Complete Production-Ready Platform

All Phases (1-7): 4,600 - 6,700 credits

- Everything polished and optimized
- SEO-ready
- Production deployment
- Good for: Professional launch

Credit Cost Factors

Credits vary based on:

- Complexity of design (simple vs. custom)
 - Number of revisions needed
 - Database complexity
 - Integration challenges
 - Bug fixes and testing iterations
-

RECOMMENDED APPROACH



Start Small, Scale Smart

Step 1: Build MVP (Phases 1-3)

Investment: 2,100-3,100 credits (~\$210-\$310 USD at current rates)

Timeline: 1-2 weeks

Outcome: Working online store

Why start here?

- Validate your business idea with real customers
- Start generating revenue quickly
- Learn what features you actually need
- Avoid over-building features nobody uses

Step 2: Launch & Test

- Add 20-50 products manually (via database tool or SQL)
- Test checkout flow with real credit card (Stripe test mode)
- Share with friends/family for feedback
- Process 5-10 test orders

Step 3: Add Admin Panel (Phase 4)

When: After you're tired of managing products via database

Investment: +800-1,200 credits

Why: Makes daily operations 10x easier

Step 4: Choose Advanced Features (Phase 5)

When: After 50-100 real orders

Investment: +500-800 credits per feature

How to decide:

- Getting product questions? → Add vehicle compatibility checker
- Want repeat customers? → Add wishlist and profiles
- Need social proof? → Add reviews and ratings

Step 5: Optimize & Scale (Phases 6-7)

When: When you're committed long-term

Investment: +700-1,100 credits

Focus: PWA, performance, SEO for growth

Pro Tips

1. **Don't build everything upfront**
 - 80% of features get 20% usage
 - Build what you need, when you need it
 2. **Validate with real users first**
 - Your assumptions about features are probably wrong
 - Let customer behavior guide development
 3. **Start with manual processes**
 - Don't automate until you've done it manually 10+ times
 - You'll learn the edge cases
 4. **Revenue first, features second**
 - A simple store making \$5k/month beats a complex store making \$0
 5. **Consider n8n after Phase 4**
 - Only when you identify repetitive manual tasks
 - Example: "I email suppliers every morning about low stock" → automate it
-

WHAT YOU'LL NEED TO PROVIDE

Before Development Starts

1. Business Information

- ☐ Business name
- ☐ ABN (Australian Business Number)
- ☐ Business address
- ☐ Contact email
- ☐ Phone number
- ☐ Trading hours

2. Payment Processing Setup

- ☐ Stripe account (create at stripe.com/au)
- Verify identity (takes 1-2 business days)
- Add bank account for payouts
- Enable "Payment Links" and "Checkout"
- ☐ GST registration status (affects tax calculations)

3. Branding Assets

- ☐ Logo (SVG or PNG with transparent background)
- ☐ Brand colors (hex codes)
- ☐ Favicon
- ☐ Sample product images (for testing)

4. Content & Policies

- ☐ About Us page content
- ☐ Contact page information
- ☐ Terms and Conditions

- ☐ Privacy Policy
- ☐ Refund/Returns Policy
- ☐ Shipping Policy

5. Product Data (Sample for MVP)

For 10-20 initial products, provide:

- ☐ Product name
- ☐ Description (detailed)
- ☐ Price (in AUD)
- ☐ Condition (New/Used)
- ☐ Category
- ☐ SKU/Part number
- ☐ Stock quantity
- ☐ 3-5 images per product
- ☐ Specifications (for parts: fits, dimensions, weight, material)

Format: Excel/CSV or Google Sheets with columns:

Title	Description	Price	Condition	Category	SKU	Stock	Image_URLs	Specifications
-------	-------------	-------	-----------	----------	-----	-------	------------	----------------

6. Shipping Information (Phase 3)

- ☐ Shipping zones (metro Sydney, regional NSW, interstate, etc.)
- ☐ Shipping rates (flat rate or weight-based?)
- ☐ Free shipping threshold (e.g., free over \$200)
- ☐ Handling time (1-2 business days?)

During Development

You'll be asked for:

- Design preferences (show example websites you like)
- Feature priorities (what's most important?)
- Feedback on mockups/prototypes
- Test account creation
- Test orders

EXTERNAL TOOLS RECOMMENDATION

Don't Use (At Least Not Initially)

n8n Workflow Automation

- **Why not now:** Adds complexity and hosting costs (\$10-20/month)
- **When to use:** After Phase 4, when you identify repetitive tasks
- **Example use cases:**
 - Auto-sync inventory with suppliers
 - Send SMS on order placed
 - Daily sales reports to email
 - Cross-post to eBay/Gumtree

Baserow / Airtable (External Databases)

- **Why not:** Slower, costs money, rate limits, harder to scale
- **Alternative:** Use PostgreSQL (free, included, faster)
- **When to use:** Never for this project (admin panel gives you same visual editing)

Zapier

- **Why not:** Expensive (\$20-100/month), limited compared to n8n
- **Alternative:** Use n8n if you need automation (self-hosted = cheaper)



Do Use (Included in Core Stack)

PostgreSQL Database

- **Free** (included in Vercel, Railway, Supabase)
- **Fast** (sub-10ms queries)
- **Scalable** (handles millions of products)
- **Powerful** (complex queries, full-text search, JSON support)

Stripe Payment Processing

- **Lowest fees** in Australia (1.75% + 30¢)
- **No monthly fee** (only per-transaction)
- **Best developer experience**
- **Handles:** Fraud detection, PCI compliance, refunds, subscriptions

Cloudinary (Image Hosting)

- **Free tier:** 25 GB storage, 25 GB bandwidth/month
- **Features:** Automatic optimization, resizing, CDN delivery
- **Alternative:** AWS S3 (cheaper at scale, more setup)

Vercel (Hosting)

- **Free tier:** Perfect for MVP (10 GB bandwidth, unlimited projects)
- **Fast:** Edge network (fast in Australia)
- **Easy:** Push to GitHub → automatic deployment
- **Alternative:** Railway (\$5-20/month, includes PostgreSQL)

Gmail SMTP / SendGrid (Email)

- **Gmail:** Free for low volume (<100 emails/day)
- **SendGrid:** Free tier (100 emails/day)
- **For:** Order confirmations, password resets



Consider Later (After MVP)

Google Analytics

- **Free**
- **When:** Phase 7 (SEO optimization)
- **Why:** Track traffic, conversions, user behavior

Hotjar / Microsoft Clarity

- **Free** heatmaps and session recordings
- **When:** After 100+ visitors
- **Why:** See where users get confused

Sentry (Error Tracking)

- **Free tier:** 5,000 errors/month
- **When:** After production launch
- **Why:** Monitor crashes and bugs

Australia Post API

- **Free** API access
- **When:** Phase 3 or 4
- **Why:** Real-time shipping rates based on size/weight

LIMITATIONS TO REMEMBER

This is a PWA, Not a Native App

What You'll Have

- Installable to phone home screen (iOS & Android)
- Full-screen experience (no browser bars)
- Fast, responsive, works offline
- Push notifications (with setup)
- Access to camera (for future AR features)

What You Won't Have

- **Not in App Store or Google Play Store**
- Users can't discover your app by searching stores
- No "Download on App Store" badge
- No automatic updates through stores
- **Limited iOS features**
- iOS Safari restricts some PWA capabilities
- Less storage quota on iOS
- No background sync on iOS (Android has it)
- **Can't access native phone features**
- No biometric login (Face ID / fingerprint) - (possible with workarounds)
- No deep system integration
- No access to contacts, calendar automatically

Installation Process

Android:

1. Visit website in Chrome
2. Popup: "Add to Home Screen"
3. Tap → App installs
4. Icon appears on home screen

iOS:

1. Visit website in Safari
2. Tap share button
3. "Add to Home Screen"

4. Tap Add → Icon appears

(More steps = lower install rate on iOS)

Performance Expectations

- **Fast:** Loads in 1-2 seconds on 4G
- **Works offline:** Can browse cached products
- **Smooth:** 60fps animations
- **But:** Still requires internet for checkout, new data

Future Native App Option

If you later want true native apps (in App Stores):

Option 1: Hire React Native developer (~\$5k-10k USD)

- Wraps your existing website
- Adds native features
- Publishes to stores

Option 2: Use Capacitor (DIY)

- Free tool that converts PWA → native app
- You handle publishing (Apple \$99/year, Google \$25 one-time)
- Less polished than custom native development

Option 3: Keep PWA only

- Many successful businesses use PWA only (Twitter, Tinder, Uber)
- Lower development cost
- One codebase = easier maintenance

SEO & Discovery

✓ **Good:** Google indexes PWAs normally

✓ **Good:** Can rank in search results

✗ **Bad:** No App Store SEO

✗ **Bad:** Must drive traffic from Google, ads, social media

STOPPING POINTS: When You Have a Functional System

You can pause development after any phase and still have a working platform. Here are the most logical stopping points:

● Stopping Point #1: After Phase 3 (Basic Working Store)

You'll have:

- Customers can browse products
- Search and filter functionality
- Complete checkout with Stripe
- Order confirmations via email

You can:

- Start selling immediately
- Process real transactions
- Get customer feedback

You'll need to:

- Manage products via database tools (SQL queries) or pgAdmin
- Manually track orders in database
- Handle customer service via email

Good for:

- Testing the market with minimal investment
- Side project or part-time business
- 0-50 orders per month

Next step decision point:

- If you're getting orders regularly → Move to Phase 4 (Admin Panel)
 - If no traction → Pause development, focus on marketing
-

Stopping Point #2: After Phase 4 (Store with Admin Management)

You'll have:

- Everything from Stopping Point #1
- Easy product management (no SQL needed)
- Order tracking dashboard
- Inventory management
- User management

You can:

- Run the store without technical skills
- Add products via web interface
- Track orders and update statuses
- Monitor sales and inventory

You'll need to:

- Decide which advanced features matter most
- Monitor customer requests for missing features

Good for:

- Full-time business operations
- 50-500 orders per month
- Team of 1-3 people

Next step decision point:

- If customers ask about reviews → Add Phase 5B
 - If mobile traffic is high → Add Phase 6 (PWA)
 - If running smoothly → Focus on marketing, pause development
-

Stopping Point #3: After Phase 6 (Full-Featured PWA)

You'll have:

- Everything from Stopping Point #2
- 2-3 advanced features (profiles, reviews, wishlist, etc.)
- Installable PWA

- Optimized mobile experience
- Offline functionality

You can:

- Compete with major e-commerce sites
- Offer app-like experience
- Drive mobile conversions

You'll need to:

- Consider Phase 7 (Polish & SEO) for growth
- Plan marketing and customer acquisition

Good for:

- Scaling business
- 500+ orders per month
- Professional operation

Next step decision point:

- If ready for growth → Complete Phase 7
 - If profitable at current scale → Maintain and optimize
 - If need automation → Add n8n
-

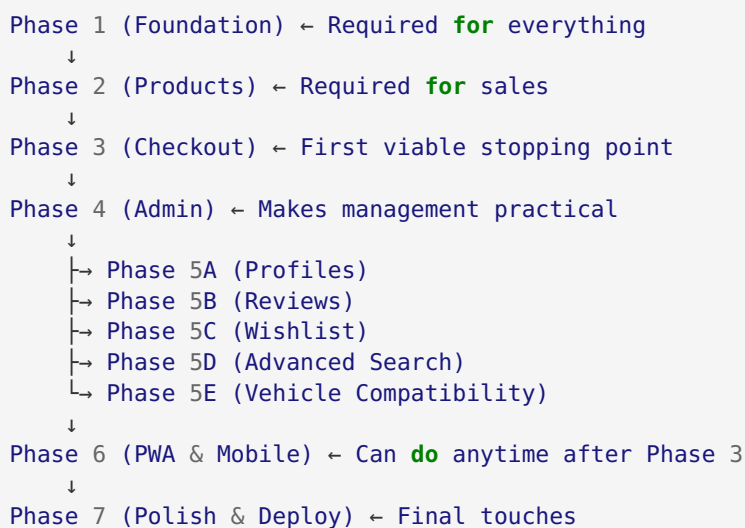
**Not Recommended Stopping Points****Don't stop after Phase 1 or 2:**

- No checkout = can't sell
- Incomplete platform confuses customers
- Bad first impression

Don't stop mid-phase:

- Half-built features are worse than no features
 - Technical debt makes resuming harder
 - Customers encounter broken functionality
-

PHASE DEPENDENCIES DIAGRAM



Key Insight: Phases 1-3 are linear (must be done in order). After Phase 4, you can pick and choose features from Phase 5 based on customer needs.

NEXT STEPS

If You're Ready to Start

1. **Review this document** and note any questions
2. **Gather required materials** (logos, product data, business info)
3. **Create Stripe account** (do this now - verification takes 1-2 days)
4. **Decide on Phase 1-3 or Phase 1-4** as initial scope
5. **Return with:** "Let's start Phase 1" and any specific requirements

If You Need to Decide

Ask yourself:

- Do I have 20+ products ready to list?
- Do I have product images and descriptions ready?
- Do I have \$300-600 USD budget for MVP? (2,100-3,100 credits)
- Can I commit 1-2 weeks for development?
- Do I have time to provide feedback during development?

If yes to all → Ready to start

If no to some → Prepare materials first, then return

Questions to Consider

- **Design style:** Prefer minimalist, colorful, professional, or rugged/automotive theme?
- **Primary audience:** DIY mechanics, professional shops, or general car owners?
- **Differentiator:** What makes your store unique? (Better prices? Rare parts? Expert advice?)
- **Vehicle focus:** All vehicles or specific makes (Toyota, Holden, Ford)?
- **Business model:** Dropshipping, inventory holding, or hybrid?

MAINTENANCE & ONGOING COSTS

After Development (Monthly Costs)

Service	Cost	What It's For
Hosting (Vercel)	\$0-20	Hosting the website (free tier usually sufficient)
Database (Railway/Supabase)	\$0-5	PostgreSQL hosting (free tier: 500MB-1GB)
Domain	\$2-3	yourstore.com.au (annual ~\$25-35)
Email (SendGrid)	\$0-15	Transactional emails (free tier: 100/day)
Image Storage (Cloudinary)	\$0-30	Product images (free tier: 25GB)
Stripe	Pay per transaction	1.75% + 30¢ per sale
SSL Certificate	\$0	Included with Vercel/Railway
Backups	\$0-10	Database backups (optional)

Total baseline: \$0-15/month for small store (under free tier limits)

Total at scale: \$50-100/month for busy store (1000+ products, 500+ orders/month)

Time Investment

- **Phase 1-3:** 10-15 hours of your time (feedback, content, testing)
 - **Ongoing:** 5-10 hours/week (customer service, product updates, order processing)
 - **With Admin Panel:** 2-5 hours/week (much easier management)
-

RISK FACTORS & MITIGATION

Technical Risks

Risk	Probability	Impact	Mitigation
Stripe integration issues	Low	High	Use Stripe Checkout (hosted), not custom flow
Database performance problems	Low	Medium	Proper indexing, use PostgreSQL (not external DBs)
Security vulnerabilities	Medium	High	Use NextAuth, validate inputs, HTTPS only
Mobile compatibility issues	Medium	Low	Test on real devices early (Phase 2)
Image loading slow	Medium	Medium	Use Cloudinary CDN, Next.js Image component

Business Risks

Risk	Probability	Impact	Mitigation
No customers after launch	Medium	High	Start marketing before launch, build email list
Product data incomplete	High	Medium	Prepare 20-50 products before development starts
Competition from eBay/Gumtree	High	Medium	Focus on better UX, expert advice, warranties
Shipping costs too high	Medium	Medium	Integrate Aus Post API for real-time rates
Stripe fees eat margins	Low	Low	1.75% is industry standard, factor into pricing

Development Risks

Risk	Probability	Impact	Mitigation
Over-building features	High	Low	Follow phased approach, stop at Phase 3 or 4 first
Scope creep	Medium	Medium	Agree on phase scope upfront, no mid-phase changes
Design indecision	Medium	Low	Provide 2-3 example sites you like upfront
Testing insufficient	Medium	Medium	Plan 2-3 hours for testing each phase

GLOSSARY: Terms You Should Know

PWA (Progressive Web App): A website that acts like a mobile app (installable, works offline)

Next.js: React framework for building fast, SEO-friendly web apps

PostgreSQL: Open-source database (stores products, orders, users)

Stripe: Payment processor (handles credit cards securely)

API: Application Programming Interface (how different software talks to each other)

SKU: Stock Keeping Unit (unique identifier for each product)

GST: Goods and Services Tax (10% in Australia)

CDN: Content Delivery Network (makes images/files load faster globally)

CRUD: Create, Read, Update, Delete (basic database operations)

Checkout Flow: The steps a customer goes through to complete a purchase

Service Worker: Background script that enables offline functionality

Webhook: Automatic notification from one service to another (e.g., Stripe tells your app “payment succeeded”)

SEO: Search Engine Optimization (making your site rank higher in Google)

Responsive Design: Website adapts to different screen sizes (mobile, tablet, desktop)

SUMMARY: Quick Reference

- ✓ **Recommended Stack:** Next.js + PostgreSQL + Stripe (no n8n, no external databases initially)
 - ✓ **Minimum Investment:** 2,100-3,100 credits for working store (Phases 1-3)
 - ✓ **First Stopping Point:** After Phase 3 (can start selling)
 - ✓ **Best Stopping Point:** After Phase 4 (easy to manage)
 - ✓ **Monthly Costs:** \$0-15 for small store, \$50-100 at scale
 - ✓ **Payment Fees:** Stripe 1.75% + 30¢ per transaction (unavoidable)
 - ✓ **What It Won't Do:** Won't be in App Stores (it's a PWA, not native app)
 - ✓ **Timeline:** 1-2 weeks for MVP, 3-4 weeks for full platform
 - ✓ **Your Preparation:** Gather 20-50 products, create Stripe account, prepare branding
-

Ready to build? Let me know which phase you'd like to start with, and we'll begin! 🚀